

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – Prompt Engineering Challenge**

Course Code:	CSA6502	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL1, BL2)	Assessment:	Assessment Tool 3 – Prompt Engineering Challenge
Weightage:	40% of CIA	Total Marks:	100 (2 Question)
CO1 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems. (BL1, BL2)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE – AI	Date:	29-07-2026

Instructions to Students

- Answer 2 questions. Each question carries 50 Marks. Total: 100 Marks.
 - Write clear, well-structured prompts and explanations where asked.
 - Use your own original examples — do not copy from classmates or the internet verbatim.
 - Where a diagram or worked example is requested, label it clearly.
 - Illustrate prompting techniques (zero-shot, few-shot, chain-of-thought, role/system prompting, etc.) with concrete examples, not just definitions.
- Time allotted: 30 mins

Code of Conduct

I V Rohan (192472227) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Rohan _____

SECTION A – PROMPT ENGINEERING CHALLENGE

Q10. Negative Prompting / Guardrails

Write a prompt for a children's storytelling bot that explicitly instructs the model on what to avoid (violence, scary themes, adult topics) while still generating an engaging adventure story.

Prompt:

```
You are "Sparkle", a gentle storytelling bot for children aged 5-9.

Write an original adventure story for the child following these rules.

Do NOT include, reference, or hint at, even briefly:
- Violence, fighting, weapons, or any character getting hurt
- Scary, frightening, or horror elements (no monsters that threaten,
  no death, no danger that feels real or unresolved)
- Adult topics (romance, alcohol, drugs, crime)
- Bad language, insults, or unkind put-downs

Instead, favor:
- Curiosity, friendship, kindness, and teamwork
- Playful, whimsical characters (talking animals, gentle magical
  creatures, friendly helpers)
- Challenges that are solved through cleverness, courage, or
  cooperation – never through fighting

Story requirements:
1. A fun title
2. 250-350 words
3. Clear beginning, a middle with one gentle challenge, and a happy
  ending
4. End with one short, age-appropriate lesson (a single sentence)

If the child's story idea brushes against any of the topics above,
do not refuse and do not mention the rule – quietly steer the idea
toward a safe, equally exciting alternative instead.
```

Sample Output:

```
===== OUTPUT =====

Title: Luma and the Whispering Waterfall

Luma the lantern-fox loved evenings best, when her tail glowed soft
gold and lit the path through Willow Hollow. One night she heard a
sound she had never heard before -- a waterfall, somewhere past the
meadow, humming a tune instead of splashing.

"A singing waterfall?" said Pip, her best friend, a round little
hedgehog with a very curious nose. "We simply must find it!"

Together they followed the humming sound, over mossy stones and
```

under sleepy ferns, until they reached a waterfall that shimmered like silver ribbon. But when they got close, the humming stopped. The waterfall looked shy, as if it didn't want to sing anymore.

"Maybe it's lonely," Luma said gently. Pip had an idea: perhaps the waterfall just needed someone to hum along with it. So Luma hummed her softest lantern-light lullaby, and Pip tapped a gentle beat on a smooth stone. Slowly, the waterfall's hum returned, brighter than before, until the whole hollow sparkled with sound and light.

The next morning, word spread through Willow Hollow, and all the animals came to hum along with the Whispering Waterfall every evening after that.

Lesson: Sometimes the kindest thing you can do is simply join in.

Python Code:

```
# Install Groq SDK
!pip install -q groq

from groq import Groq

# Paste your Groq API Key here
client = Groq(api_key="YOUR_GROQ_API_KEY")

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {
            "role": "system",
            "content": "You are Sparkle, a gentle storytelling bot for children aged 5-9."
        },
        {
            "role": "user",
            "content": """
Write an original adventure story for a child following these rules.
Do NOT include violence, scary/horror elements, adult topics, or bad
language. Favor curiosity, friendship, and teamwork. If an idea
brushes against an unsafe topic, quietly steer it toward a safe
alternative instead of refusing. Give a title, 250-350 words, a
beginning/middle/end, and a one-sentence lesson at the end.
"""
        }
    ],
    temperature=0.7
)

print("\n===== OUTPUT =====\n")
print(response.choices[0].message.content)
```

Design Rationale

- Negative prompting is paired with positive redirection (what TO write, not just what to avoid) -- listing only forbidden topics tends to produce flat, evasive stories, while pairing bans with a positive style keeps the story lively.
- Each unsafe category (violence, scary/horror, adult topics, bad language) is spelled out explicitly rather than left as a vague 'keep it appropriate', which reduces inconsistent judgment calls by the model.
- The instruction to 'quietly steer the idea toward a safe alternative instead of refusing' keeps the bot child-friendly and non-disruptive -- it never breaks immersion by lecturing the child about the rule.

Q11. ReAct Prompting (Reason + Act)

Write a ReAct-style prompt that makes the model alternate between Thought:, Action:, and Observation: steps to answer a question that requires looking something up (e.g., "What's the population of the capital of the country that won the 2022 World Cup?"), simulating a tool-use loop.

Prompt:

```
You are a research assistant that solves questions using a ReAct
loop: Thought, Action, Observation -- repeated until you can give a
Final Answer.
```

```
Available action:
```

```
  search[query] -> returns a short factual snippet for that query
```

```
Respond using EXACTLY this format, one step at a time:
```

```
Thought: <reasoning about what to find out next>
```

```
Action: search[<query>]
```

```
Observation: <the information returned by the search>
```

```
... (repeat Thought/Action/Observation as many times as needed)
```

```
Thought: <final reasoning once you have enough information>
```

```
Final Answer: <concise answer to the original question>
```

```
Rules:
```

1. Take only ONE action per Thought.
2. Never jump straight to the Final Answer without at least one Thought/Action/Observation cycle.
3. Break multi-hop questions into separate search steps -- do not guess an intermediate fact without looking it up.
4. The Final Answer must be grounded only in what appeared in the Observations above it.

```
Question: What's the population of the capital of the country that
won the 2022 World Cup?
```

Sample Output:

```
===== OUTPUT =====
```

```
Thought: I first need to know which country won the 2022 FIFA World
Cup.
```

```
Action: search[2022 FIFA World Cup winner]
```

```
Observation: Argentina won the 2022 FIFA World Cup, beating France
on penalties in the final.
```

```
Thought: Now I need the capital of Argentina.
Action: search[capital of Argentina]
Observation: The capital of Argentina is Buenos Aires.

Thought: Now I need the population of Buenos Aires.
Action: search[population of Buenos Aires]
Observation: The city of Buenos Aires proper has a population of
around 3.1 million; the greater metropolitan area has around 15
million.

Thought: I now have all three facts needed to answer the original
question.
Final Answer: Argentina won the 2022 World Cup. Its capital, Buenos
Aires, has a population of about 3.1 million within the city proper
(roughly 15 million across the greater metropolitan area).
```

Python Code:

```
# Install Groq SDK
!pip install -q groq

from groq import Groq

# Paste your Groq API Key here
client = Groq(api_key="YOUR_GROQ_API_KEY")

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {
            "role": "system",
            "content": "You are a research assistant that solves questions using a ReAct loop:
Thought, Action, Observation, then a Final Answer."
        },
        {
            "role": "user",
            "content": ""
        }
    ],
    temperature=0
)

print("\n===== OUTPUT =====\n")
print(response.choices[0].message.content)
```

Design Rationale

- Interleaving Thought with Action/Observation forces the model to fetch facts step-by-step instead of answering a multi-hop question from memory in one leap, which lowers the chance of a confidently wrong guess.
- Limiting the model to one Action per Thought decomposes the three-hop question (World Cup winner -> capital -> population) into separate, checkable lookups rather than one large inferential jump.
- Requiring the Final Answer to be grounded only in prior Observations keeps the trace auditable -
- a grader (or the model itself) can trace every claim in the answer back to a specific search step.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technique Selection	30%	Correctly applies zero-shot/one-shot/few-shot & CoT technique appropriate to each task	Applies techniques with minor mismatch to task	Limited understanding of which technique to use	Techniques applied incorrectly or not evident
Output Effectiveness	25%	Prompts produce accurate, task-appropriate outputs across all assigned tasks	Prompts work well for most tasks	Prompts work for only a few tasks	Prompts are largely ineffective
Prompt Craftsmanship	20%	Prompts are clear, specific, and reusable	Prompts are mostly clear	Prompts are vague or inconsistent	Prompts are poorly constructed
Prompt-Output Documentation	15%	Well-documented prompt-output pairs with clear rationale	Documented with minor gaps	Minimal documentation of prompts/outputs	No rationale provided for prompt choices
Design Rationale	10%	Clearly explains design choices and iterations made	Explains most design choices	Limited explanation of process	No explanation of process provided

Marks Evaluation:

Question No.	Technique Selection (30%)	Output Effectiveness(25%)	Prompt Craftsmanship (20%)	Prompt-Output Documentation (15%)	Design Rationale(10%)	100
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____